MASM64 & x86-64 Assembly

Computer Architecture — Pre-Lecture Study Notes

Compiled for 5th Semester BSCS · University of Balochistan

These notes cover everything you need to walk into tomorrow's lecture with confidence: architecture concepts, registers, memory model, instruction set, program structure, and working code examples.

1. What is x86-64 Architecture?

x86-64 (also called AMD64 or Intel 64) is a 64-bit extension of the classic x86 instruction set architecture. It is what runs inside your Ryzen 5 7430U and virtually every modern PC.

Key Characteristics

Feature	x86 (32-bit)	x86-64 (64-bit)
General-purpose registers	8 (EAX–ESP)	16 (RAX–R15)
Register width	32 bits	64 bits
Addressable memory	4 GB (2^{32})	16 EB (2^{64} theoretical)
Floating-point	x87 / SSE	SSE2 mandatory + AVX
Calling convention	Varies / stack heavy	Register-first (RCX,RDX,R8,R9)
Pointer size	4 bytes	8 bytes

Why MASM64?

- MASM (Microsoft Macro Assembler) is the assembler that ships with VS and the Windows SDK.
- The 64-bit version (ml64.exe) assembles x86-64 code and integrates with Visual Studio / VS Code.
- It is the industry-standard tool on Windows for systems programming, OS development, and reverse engineering.

 You already set up MASM64 in VS Code on your machine after the Windows reset — so the toolchain is ready.

2. The Register File — Your CPU's Fast Variables

Registers are tiny storage cells inside the CPU. They are orders of magnitude faster than RAM. In x86-64 there are 16 general-purpose registers (GPRs), each 64 bits wide.

General-Purpose Registers

64-bit	32-bit	16-bit	8-bit (low)	Conventional Use
RAX	EAX	AX	AL	Accumulator / return value
RBX	EBX	BX	BL	Base / callee-saved
RCX	ECX	CX	CL	Counter / 1st arg (Windows)
RDX	EDX	DX	DL	Data / 2nd arg (Windows)
RSI	ESI	SI	SIL	Source index
RDI	EDI	DI	DIL	Destination index
RSP	ESP	SP	SPL	Stack pointer (DO NOT touch carelessly)
RBP	EBP	BP	BPL	Base/frame pointer
R8	R8D	R8W	R8B	3rd arg (Windows)
R9	R9D	R9W	R9B	4th arg (Windows)
R10–R15	R10D–R15D	R10W–R15W	R10B–R15B	Caller-saved scratch

 Writing to EAX (32-bit) automatically ZERO-EXTENDS into RAX — the upper 32 bits are cleared. Writing to AX (16-bit) does NOT zero the upper bits. This asymmetry trips up every beginner.

Special-Purpose Registers

Register	Purpose
RIP	Instruction Pointer — points to the next instruction
RFLAGS	Status flags set by arithmetic/logic instructions
XMM0–XMM15	128-bit SSE/floating-point registers
YMM0–YMM15	256-bit AVX registers (lower 128 = XMM)

RFLAGS — The Status Register

RFLAGS is a 64-bit register where individual bits signal the outcome of the last operation:

Flag	Symbol	Meaning
Zero Flag	ZF	Result was zero (used by JE/JZ)
Carry Flag	CF	Unsigned overflow / borrow
Sign Flag	SF	Result was negative
Overflow Flag	OF	Signed overflow
Parity Flag	PF	Low byte has even number of 1-bits
Direction Flag	DF	String operation direction (0=forward)

3. Memory Model & Segments

Virtual Address Space

Each process gets its own private 64-bit virtual address space (though in practice Windows uses only ~128 TB). The OS/CPU translates virtual addresses to physical RAM transparently.

Classical Memory Segments

Segment	Direction	Content
Text (.code)	→ fixed	Machine instructions (read-only at runtime)
Data (.data)	→ fixed	Initialized global/static variables
BSS (.data?)	→ fixed	Uninitialized globals (zero-filled by OS)
Stack	← grows down	Local variables, return addresses, saved registers
Heap	→ grows up	Dynamic allocations (malloc / HeapAlloc)

The Stack

The stack is a LIFO (Last-In First-Out) region of memory. RSP always points to the top (lowest address, since it grows downward).

- PUSH value → RSP -= 8, then writes value at [RSP]
- POP dest → reads value from [RSP], then RSP += 8
- CALL target → PUSHes return address, then JMPs to target
- RET → POPs return address into RIP

 Windows x64 ABI requires RSP to be 16-byte aligned BEFORE a CALL instruction. If your function does not realign, SSE instructions will crash.

4. Data Types & Directives

Data Size Keywords

MASM Keyword	Bytes	Bits	C Equivalent
BYTE / DB	1	8	unsigned char
WORD / DW	2	16	unsigned short
DWORD / DD	4	32	unsigned int
QWORD / DQ	8	64	unsigned long long / pointer
OWORD / DDQ	16	128	__m128 (SSE)
REAL4	4	32	float
REAL8	8	64	double

Declaring Data in .data Section

```
.data
    myByte    BYTE    42
    myWord    WORD    1000
    myDword   DWORD   0DEADBEEFh      ; hex literal
    myQword   QWORD   1234567890ABCDEFh
    myFloat   REAL4   3.14
    myStr     BYTE    'Hello, World!', 0  ; null-terminated
    arr       DWORD   10, 20, 30, 40, 50  ; array of 5
    buf       BYTE    128 DUP(0)           ; 128-byte zeroed buffer
```

Pointer / Memory Access Syntax

```
mov rax, [rbx]          ; load 64-bit value from address in RBX
mov eax, [rbx]          ; load 32-bit value
mov al,  BYTE PTR [rbx] ; load 1 byte
mov [rbx], rcx          ; store RCX to address in RBX
mov rax, [rbx + 8]      ; load from RBX+8  (struct field / array element)
mov rax, [rbx + rcx*8]  ; scaled index addressing
```

5. Core Instruction Set

5.1 Data Movement

Instruction	Syntax	Description
MOV	mov dst, src	Copy src into dst (most common instruction)
MOVZX	movzx dst, src	Move with zero-extension (smaller → larger)
MOVSX	movsx dst, src	Move with sign-extension
XCHG	xchg a, b	Swap two operands atomically
LEA	lea dst, [expr]	Load Effective Address — computes address, stores it
PUSH	push src	Decrement RSP, store src on stack
POP	pop dst	Load from stack top, increment RSP

 *LEA is often used for fast arithmetic: 'lea rax, [rbx + rcx*4 + 8]' computes the value without touching memory or flags — a common compiler trick.*

5.2 Arithmetic

Instruction	Syntax	Notes
ADD	add dst, src	dst = dst + src; sets flags
SUB	sub dst, src	dst = dst - src; sets flags
MUL	mul src	RDX:RAX = RAX * src (unsigned)
IMUL	imul dst, src	dst = dst * src (signed, two-operand form)
DIV	div src	RDX=remainder, RAX=quotient; dividend in RDX:RAX
IDIV	idiv src	Signed division
INC	inc dst	dst++ (does NOT set CF)
DEC	dec dst	dst-- (does NOT set CF)
NEG	neg dst	dst = -dst (two's complement)
ADC	adc dst, src	Add with carry (CF): multi-precision addition
SBB	sbb dst, src	Subtract with borrow

5.3 Bitwise / Logical

Instruction	Effect
AND dst, src	Bitwise AND — clears bits

Instruction	Effect
OR dst, src	Bitwise OR — sets bits
XOR dst, src	Bitwise XOR — flips bits; xor rax,rax is fast register-clear
NOT dst	Bitwise NOT (one's complement)
SHL dst, count	Shift left by count (multiply by 2^count)
SHR dst, count	Shift right logical (unsigned divide by 2^count)
SAR dst, count	Shift right arithmetic (preserves sign bit)
ROL / ROR dst, count	Rotate left / right
BT / BTS / BTR	Bit test / test-and-set / test-and-reset

5.4 Comparison & Flags

```
cmp rax, rbx    ; computes RAX - RBX but discards result, only sets flags
test rax, rax   ; computes RAX & RAX; ZF=1 if RAX==0  (faster than cmp rax,0)
test rax, 1     ; checks if bit 0 is set
```

5.5 Control Flow — Jumps & Branches

Instruction	Condition	Flag(s) checked
JMP target	Unconditional	—
JE / JZ	Equal / Zero	ZF=1
JNE / JNZ	Not equal / Not zero	ZF=0
JG / JNLE	Greater (signed)	ZF=0 and SF=OF
JGE / JNL	Greater or equal (signed)	SF=OF
JL / JNGE	Less (signed)	SF≠OF
JLE / JNG	Less or equal (signed)	ZF=1 or SF≠OF
JA / JNBE	Above (unsigned)	CF=0 and ZF=0
JB / JNAE	Below (unsigned)	CF=1
JRCXZ	RCX == 0	RCX directly
LOOP target	Decrement RCX, jump if RCX≠0	RCX

6. MASM64 Program Structure

Every MASM64 program follows a fixed skeleton. Here is the minimal template you must know:

```
; ─── masm64_template.asm ──────────────────────────────
OPTION CASEMAP:NONE          ; make labels case-sensitive

INCLUDE \masm32\include64\masm64rt.inc   ; common macros/includes

.data
    msg  BYTE  'Hello from MASM64!', 0Ah, 0   ; 0Ah = newline

.code

main PROC
    sub  rsp, 28h             ; shadow space (32 bytes) + align to 16

    lea  rcx, msg             ; 1st arg = pointer to string
    call printf               ; call C runtime printf

    xor  ecx, ecx             ; exit code = 0
    call ExitProcess
main ENDP

END
```

Shadow Space — The 32-Byte Rule

Windows x64 calling convention REQUIRES the caller to reserve 32 bytes of 'shadow space' (also called home space) on the stack before any CALL. Even if the callee takes fewer than 4 arguments, you still allocate 32 bytes.

```
sub rsp, 28h    ; 0x28 = 40 bytes: 32 shadow + 8 to keep RSP 16-byte aligned
; ... do work ...
add rsp, 28h    ; restore before RET
```

 Forgetting shadow space is the #1 cause of mysterious crashes in MASM64. Always do 'sub rsp, 28h' at function entry.

Procedures

```
myProc PROC
    ; prologue
    push rbp
    mov  rbp, rsp
    sub  rsp, 20h           ; local variables + shadow space

    ; body ...
    mov  rax, 42            ; return value goes in RAX
```

```
    ; epilogue
    add  rsp, 20h
    pop  rbp
    ret
myProc ENDP
```

7. Windows x64 Calling Convention (ABI)

When your assembly calls Windows API functions or C runtime functions, it must follow the Microsoft x64 ABI precisely.

Argument Passing

Argument #	Register	Notes
1st	RCX	Integer / pointer
2nd	RDX	Integer / pointer
3rd	R8	Integer / pointer
4th	R9	Integer / pointer
5th+	Stack	Pushed right-to-left, above shadow space
Float 1st–4th	XMM0–XMM3	Floating-point args shadow the integer registers

Return Values

- Integer / pointer returns go in RAX.
- 64-bit floats return in XMM0.

Register Preservation (Caller vs Callee)

Caller-saved (volatile)	Callee-saved (non-volatile)
RAX, RCX, RDX, R8, R9, R10, R11	RBX, RBP, RDI, RSI, RSP, R12–R15
XMM0–XMM5	XMM6–XMM15

Callee-saved means: if your function uses RBX, you MUST push it at entry and pop it before returning.

8. Working Code Examples

Example 1 — Sum of Array

Sum 5 integers stored in .data and print the result.

```
OPTION CASEMAP:NONE
INCLUDE \masm32\include64\masm64rt.inc

.data
    nums  DWORD   10, 20, 30, 40, 50
    fmt   BYTE    'Sum = %d', 0Ah, 0

.code
main PROC
    sub   rsp, 28h

    xor   rax, rax          ; rax = accumulator = 0
    xor   rcx, rcx          ; rcx = index = 0
    lea   rdx, nums         ; rdx = base address

sumLoop:
    mov   eax, eax          ; clear upper 32-bits of rax (safety)
    add   eax, [rdx + rcx*4]; eax += nums[rcx]
    inc   rcx
    cmp   rcx, 5
    jl    sumLoop

    ; print: printf(fmt, sum)
    mov   r8,  rax          ; 3rd arg = sum (move before clobbering)
    lea   rdx, fmt          ; wait — actually rcx=fmt, rdx=sum
    mov   rdx, rax
    lea   rcx, fmt
    call  printf

    xor   ecx, ecx
    call  ExitProcess
main ENDP
END
```

Example 2 — Factorial (Recursive)

```
; factorial(n) in RCX -> result in RAX
factorial PROC
    push  rbp
    mov   rbp, rsp
    sub   rsp, 30h

    cmp   rcx, 1
    jle   baseCase

    mov   [rsp+20h], rcx    ; save n in shadow space
```

```
    dec   rcx
    call  factorial         ; factorial(n-1) -> RAX
    mov   rcx, [rsp+20h]    ; restore n
    imul  rax, rcx          ; n * factorial(n-1)
    jmp   done

baseCase:
    mov   rax, 1

done:
    add   rsp, 30h
    pop   rbp
    ret
factorial ENDP
```

Example 3 — String Length (strlen)

```
; Input:  RCX = pointer to null-terminated string
; Output: RAX = length (not counting null)
myStrlen PROC
    sub  rsp, 28h
    xor  rax, rax            ; count = 0

nextChar:
    cmp  BYTE PTR [rcx + rax], 0
    je   done
    inc  rax
    jmp  nextChar

done:
    add  rsp, 28h
    ret
myStrlen ENDP
```

9. MASM64 Assembler Directives Quick Reference

Directive	Purpose	Example
PROC / ENDP	Define a procedure	main PROC … main ENDP
.code	Start code segment	.code
.data	Start initialized data segment	.data
.data?	Start uninitialized data (BSS)	.data?
.const	Read-only data (string literals)	.const
EXTERN	Reference external symbol	EXTERN printf:PROC
PUBLIC	Export symbol	PUBLIC myFunc
INCLUDE	Include another file	INCLUDE masm64rt.inc
COMMENT	Multi-line comment	COMMENT ! ... !
ALIGN	Align next item	ALIGN 16
EQU	Named constant	BUFSIZE EQU 256
TEXTEQU	Text macro	CRLF TEXTEQU <0Dh, 0Ah>
MACRO / ENDM	Define a macro	myMacro MACRO … ENDM
IF / ENDIF	Conditional assembly	IF DEBUG … ENDIF
DUP	Repeat initializer	buf BYTE 256 DUP(0)
OFFSET	Offset of label	lea rcx, OFFSET msg ; or just lea rcx, msg
SIZEOF	Size in bytes	mov ecx, SIZEOF buf
LENGTHOF	Number of elements	mov ecx, LENGTHOF arr

10. Debugging & Common Mistakes

Building and Running in VS Code

```
; tasks.json snippet for ml64
"command": "ml64.exe"
"args": ["/c", "/Zi", "${file}", "/Fo",
"${fileDirname}\\${fileBasenameNoExtension}.obj"]

; then link with:
"link.exe /DEBUG /SUBSYSTEM:CONSOLE /ENTRY:main ${obj} kernel32.lib msvcrt.lib"
```

Top 10 Beginner Mistakes

#	Mistake	Fix
1	Forgetting 'sub rsp, 28h' shadow space	Always do it before any CALL
2	Writing to EAX expecting upper 32 bits preserved	EAX write zero-extends to RAX — expected for 32-bit ops
3	Using byte/word registers in 64-bit address math	Always use 64-bit registers for addresses
4	Confusing MOV and LEA	LEA computes address; MOV reads memory
5	Wrong jump type (signed vs unsigned)	Use JL/JG for signed, JB/JA for unsigned
6	Unbalanced stack (push without matching pop)	Check RSP before/after every function
7	Clobbering callee-saved registers	Push RBX, RBP, RSI, RDI at proc entry
8	Calling convention argument order wrong	RCX, RDX, R8, R9 then stack
9	Accessing array with wrong element size	DWORD array: [base + index*4], QWORD: *8
10	Missing 'END' at end of file	Every MASM file needs END (optionally END main)

11. Quick Cheat Sheet — Everything on One Page

Register Cheat Sheet

```
64-bit  32-bit  16-bit  8-bit   Role
RAX     EAX     AX      AL      Return value / accumulator
RBX     EBX     BX      BL      Callee-saved base
RCX     ECX     CX      CL      1st arg / counter
RDX     EDX     DX      DL      2nd arg / data
RSI     ESI     SI      SIL     Source
RDI     EDI     DI      DIL     Destination
RSP     ESP     SP      SPL     Stack pointer
RBP     EBP     BP      BPL     Frame pointer
R8      R8D     R8W     R8B     3rd arg
R9      R9D     R9W     R9B     4th arg
R10-R15 ...                     Caller-saved scratch
```

Most-Used Instructions

```
mov  dst, src          ; copy
add  dst, src          ; dst += src
sub  dst, src          ; dst -= src
imul dst, src          ; dst *= src
xor  dst, dst          ; fast zero
inc  dst  /  dec dst   ; ±1
push src  /  pop dst   ; stack
lea  dst, [expr]       ; load address
cmp  a, b              ; set flags (a-b)
test a, b              ; set flags (a&b)
jmp/je/jne/jl/jg/jle/jge/jb/ja  ; branches
call proc              ; call (pushes RIP)
ret                    ; return (pops RIP)
```

Program Skeleton (copy-paste ready)

```
OPTION CASEMAP:NONE
INCLUDE \masm32\include64\masm64rt.inc

.data
    ; your variables here

.code
main PROC
    sub rsp, 28h          ; shadow space

    ; your code here

    xor ecx, ecx
    call ExitProcess
main ENDP
END
```

Best of luck in tomorrow's lecture! In Shaa Allah it goes well □